

用户:帮我介绍一下FFT与NTT

AI:当然。FFT 和 NTT 都是用来**快速做多项式乘法/卷积**的算法家族。它们的核心思想相似：把多项式从“系数表示”转换到“点值表示”，逐点相乘，再转换回来。

1. FFT 是什么

FFT，全称 Fast Fourier Transform，快速傅里叶变换。

它用于快速计算离散傅里叶变换 DFT：

$$A_k = \sum_{j=0}^{n-1} a_j * \omega_n^{jk}$$

其中 ω_n 是复数单位根，比如：

$$\omega_n = e^{2\pi i/n}$$

普通 DFT 需要 $O(n^2)$ ，FFT 利用单位根的对称性，把复杂度降到：

$$O(n \log n)$$

在算法竞赛和工程里，FFT 常用于：

- 大整数乘法
- 多项式乘法
- 信号处理
- 图像处理
- 卷积计算

例如两个多项式：

$$A(x) = a_0 + a_1x + a_2x^2$$

$$B(x) = b_0 + b_1x + b_2x^2$$

朴素乘法是两层循环，复杂度 $O(n^2)$ ；用 FFT 可以做到 $O(n \log n)$ 。

2. NTT 是什么

NTT，全称 Number Theoretic Transform，数论变换。

它可以理解为“在模意义下的 FFT”。FFT 使用复数单位根，而 NTT 使用模数域里的原根。

NTT 的形式类似：

$$A_k = \sum_{j=0}^{n-1} a_j * g^{jk} \bmod p$$

其中：

- p 是质数模数
- g 是模 p 意义下的合适单位根
- 所有运算都在整数取模下进行

NTT 的优势是：

- 没有浮点误差
- 结果精确
- 很适合算法竞赛和密码学相关计算

常见 NTT 模数：

$$998244353 = 119 * 2^{23} + 1$$

它很常用，因为它支持长度最高到 2^{23} 的 NTT，并且原根通常取 3。

3. FFT 和 NTT 的区别

对比项	FFT	NTT
运算对象	复数	模整数
是否有误差	有浮点误差	精确
常用场景	信号处理、大数乘法	竞赛、多项式、模运算
单位根	复数单位根	模意义下的原根
模数限制	无	需要特殊质数
实现难度	要处理精度	要处理模数和原根

4. 它们为什么能加速乘法

多项式乘法本质上是卷积：

$$c_k = \sum a_i * b_j \quad \text{where } i + j = k$$

直接算每个 c_k 是 $O(n^2)$ 。

FFT/NTT 的套路是：

系数表示 $\rightarrow$ 点值表示 $\rightarrow$ 点值逐项相乘 $\rightarrow$ 系数表示

也就是：

```
A, B
=> FFT(A), FFT(B)
=> FFT(A) * FFT(B)
=> inverse FFT
=> C
```

NTT 同理，只是所有步骤都在模数下做。

一句话总结

FFT 是用复数做快速卷积，速度快但可能有精度误差；NTT 是用模整数做快速卷积，结果精确，但需要合适的质数模数。两者思想几乎一致，都是把多项式乘法从 $O(n^2)$ 优化到 $O(n \log n)$ 。

用户:详细介绍一下NTT的原理，然后给我举一个n=8时NTT蝶形变换的例子，包括位反转的过程，我想知道这是怎么运行的

下面用“原理 + 完整手算例子”的方式讲。我们选一个很小的模数，方便看清楚 NTT 是怎么跑的。

一、NTT 的核心思想

NTT，全称 Number Theoretic Transform，可以理解成“在模整数域里的 FFT”。

普通多项式乘法：

$$A(x) = a_0 + a_1x + \dots + a_{(n-1)}x^{(n-1)}$$

$$B(x) = b_0 + b_1x + \dots + b_{(n-1)}x^{(n-1)}$$

直接乘是卷积：

$$c_k = \sum a_i * b_j \quad \text{where } i + j = k$$

复杂度是 $O(n^2)$ 。

NTT 的想法是：

系数表示 $\rightarrow$ 点值表示 $\rightarrow$ 点值逐项相乘 $\rightarrow$ 系数表示

也就是把多项式在一些特殊点上求值。这些特殊点不是复数单位根，而是模 p 意义下的单位根。

如果模数 p 满足：

$$n \mid (p - 1)$$

那么模 p 下可以找到一个 n 次单位根 ω ，满足：

$$\omega^n \equiv 1 \pmod{p}$$

$$\omega^k \not\equiv 1 \pmod{p}, \quad 0 < k < n$$

NTT 定义为：

$$A_k = \sum_{j=0}^{n-1} a_j * \omega^{(j*k)} \pmod{p}$$

这和 DFT 形式几乎一样，只是所有运算都在模 p 下完成。

二、为什么可以蝶形加速

设 n 是 2 的幂。把多项式按下标奇偶拆开：

$$\begin{aligned} A(x) &= a_0 + a_1x + a_2x^2 + a_3x^3 + \dots \\ &= E(x^2) + x * O(x^2) \end{aligned}$$

其中：

$$E(x) = a_0 + a_2x + a_4x^2 + \dots$$

$$O(x) = a_1 + a_3x + a_5x^2 + \dots$$

要求：

$$A(\omega^k)$$

有：

$$A(\omega^k)$$

$$= E(\omega^{2k}) + \omega^k * O(\omega^{2k})$$

而另一半：

$$A(\omega^{k + n/2})$$

$$= E(\omega^{2k}) - \omega^k * O(\omega^{2k})$$

因为：

$$\omega^{n/2} \equiv -1 \pmod{p}$$

所以每次可以用两个小 NTT 的结果合成两个大 NTT 的结果：

$$u = \text{even_part}$$

$$v = w * \text{odd_part}$$

$$\text{上半部分} = u + v$$

$$\text{下半部分} = u - v$$

这就是蝶形变换。

三、选一个 $n = 8$ 的例子

我们取：

$p = 17$
 $n = 8$

模 17 下的一个原根是 3。因为：

$17 - 1 = 16$

长度为 8 时，我们需要一个 8 次单位根：

$\omega = 3^{((17 - 1) / 8)} = 3^2 = 9 \bmod 17$

验证一下：

$9^1 \equiv 9$
 $9^2 \equiv 13$
 $9^3 \equiv 15$
 $9^4 \equiv 16 \equiv -1$
 $9^8 \equiv 1$

所以 9 是模 17 下的 8 次原根。

我们对这个数组做 NTT：

$a = [1, 2, 3, 4, 5, 6, 7, 8]$

四、位反转过程

因为 $n = 8$ ，下标需要 3 位二进制：

原下标	二进制	位反转	新下标
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6

原下标	二进制	位反转	新下标
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

原数组：

[1, 2, 3, 4, 5, 6, 7, 8]

位反转后，位置 i 放原来的 rev(i)：

[1, 5, 3, 7, 2, 6, 4, 8]

也可以理解成交换：

swap(1, 4)
swap(3, 6)

得到：

[1, 5, 3, 7, 2, 6, 4, 8]

这一步的目的是让后面的蝶形可以原地、连续地做。

五、第一层蝶形：len = 2

这一层每 2 个数一组。

$w_{len} = \omega^{(8 / 2)} = \omega^4 = 9^4 = 16 \equiv -1$

但是每组只有一个蝶形，实际用到的 $w = 1$ 。

初始：

[1, 5, 3, 7, 2, 6, 4, 8]

每组做：

$u = a[i]$

$v = a[i + 1]$

$a[i] = u + v$

$a[i + 1] = u - v$

全部模 17。

第一组：

$[1, 5] \rightarrow [1 + 5, 1 - 5] = [6, -4] = [6, 13]$

第二组：

$[3, 7] \rightarrow [10, -4] = [10, 13]$

第三组：

$[2, 6] \rightarrow [8, -4] = [8, 13]$

第四组：

$[4, 8] \rightarrow [12, -4] = [12, 13]$

得到：

[6, 13, 10, 13, 8, 13, 12, 13]

六、第二层蝶形：len = 4

这一层每 4 个数一组，每组有 2 个蝶形。

$$wlen = \omega^{(8 / 4)} = \omega^2 = 9^2 = 13$$

每组内的旋转因子依次是：

$$w = 1, 13$$

当前数组：

$$[6, 13, 10, 13, 8, 13, 12, 13]$$

第一组：

$$[6, 13, 10, 13]$$

第一个蝶形：

$$u = 6$$

$$v = 10 * 1 = 10$$

结果：

$$6 + 10 = 16$$

$$6 - 10 = -4 = 13$$

第二个蝶形：

$$u = 13$$

$$v = 13 * 13 = 169 \equiv 16$$

结果：

$$13 + 16 = 29 \equiv 12$$

$$13 - 16 = -3 \equiv 14$$

所以第一组变成：

$$[16, 12, 13, 14]$$

第二组：

[8, 13, 12, 13]

第一个蝶形：

$$u = 8$$

$$v = 12 * 1 = 12$$

$$8 + 12 = 20 \equiv 3$$

$$8 - 12 = -4 \equiv 13$$

第二个蝶形：

$$u = 13$$

$$v = 13 * 13 = 169 \equiv 16$$

$$13 + 16 = 29 \equiv 12$$

$$13 - 16 = -3 \equiv 14$$

第二组变成：

[3, 12, 13, 14]

得到：

[16, 12, 13, 14, 3, 12, 13, 14]

七、第三层蝶形：len = 8

这一层整个数组是一组。

$$wlen = \omega^{(8 / 8)} = \omega = 9$$

组内旋转因子依次是：

$$w = 1, 9, 9^2, 9^3$$

$$= 1, 9, 13, 15$$

当前数组：

[16, 12, 13, 14, 3, 12, 13, 14]

现在前 4 个和后 4 个做蝶形。

第 0 个蝶形：

$$u = 16$$

$$v = 3 * 1 = 3$$

$$16 + 3 = 19 \equiv 2$$

$$16 - 3 = 13$$

第 1 个蝶形：

$$u = 12$$

$$v = 12 * 9 = 108 \equiv 6$$

$$12 + 6 = 18 \equiv 1$$

$$12 - 6 = 6$$

第 2 个蝶形：

$$u = 13$$

$$v = 13 * 13 = 169 \equiv 16$$

$$13 + 16 = 29 \equiv 12$$

$$13 - 16 = -3 \equiv 14$$

第 3 个蝶形：

$$u = 14$$

$$v = 14 * 15 = 210 \equiv 6$$

$$14 + 6 = 20 \equiv 3$$

$$14 - 6 = 8$$

最终结果：

```
NTT([1,2,3,4,5,6,7,8])  
= [2, 1, 12, 3, 13, 6, 14, 8]
```

八、整体流程总结

对于长度 8 的迭代 NTT：

原数组

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

位反转

```
[1, 5, 3, 7, 2, 6, 4, 8]
```

len = 2

```
[6, 13, 10, 13, 8, 13, 12, 13]
```

len = 4

```
[16, 12, 13, 14, 3, 12, 13, 14]
```

len = 8

```
[2, 1, 12, 3, 13, 6, 14, 8]
```

NTT 的代码结构通常就是：

```
for (int len = 2; len <= n; len <= 1) {  
    wlen = pow(omega, n / len);  
    for (int i = 0; i < n; i += len) {  
        w = 1;  
        for (int j = 0; j < len / 2; j++) {  
            u = a[i + j];  
            v = a[i + j + len / 2] * w % mod;  
  
            a[i + j] = (u + v) % mod;  
            a[i + j + len / 2] = (u - v + mod) % mod;  
  
            w = w * wlen % mod;  
        }  
    }  
}
```

一句话抓住它：位反转先把递归 FFT/NTT 的叶子顺序排好，然后每一层用单位根做蝶形合并，长度从 2、4、8 一路翻倍，最后得到在 $\omega^0, \omega^1, \dots, \omega^7$ 这些点上的取值。

用户:大模数下的NTT的难点在哪，模数超出int的范围

大模数下 NTT 的主要难点不在“NTT 思想变了”，而在**模乘会溢出**、**可用原根/模数受限制**、以及**性能会明显变敏感**。

1. 最大的问题：乘法溢出

普通 NTT 里大量出现：

```
v = a[i + j + len / 2] * w % mod;
w = w * wlen % mod;
```

如果 `mod` 在 `int` 范围内，比如 998244353，那么两个数相乘最大约：

$$1e9 * 1e9 = 1e18$$

用 `long long` 还能装下。

但如果模数超过 `int`，甚至接近 `long long`：

$$\text{mod} \approx 1e18$$

那么乘法会变成：

$$1e18 * 1e18 = 1e36$$

这远远超过 `uint64_t` 或 `long long` 的范围。

所以大模数 NTT 最先遇到的是：**不能直接写 `a * b % mod`**。

2. 常见解决方法

如果模数小于 2^{63} ，在 C++ 里常用：

`__int128`

例如：

```
long long mul_mod(long long a, long long b, long long mod) {  
    return (__int128)a * b % mod;  
}
```

这能处理：

`a, b, mod <  $2^{63}$`

因为 `__int128` 能装下 2^{126} 级别的中间乘积。

如果模数接近或超过 64 位，就不能只靠 `__int128`，需要：

- Barrett reduction
- Montgomery multiplication
- 多精度整数
- 拆位乘法
- 使用多个小 NTT 模数做 CRT 合并

在算法竞赛中，最常见、最实用的是最后一种。

3. 用多个小模数 CRT 合并

如果目标模数很大，或者目标模数本身不适合做 NTT，可以选几个 NTT-friendly 小质数，例如：

998244353
1004535809
469762049

分别在这些模数下做 NTT，得到结果：

$c \bmod p_1$
 $c \bmod p_2$
 $c \bmod p_3$

然后用 CRT，中国剩余定理，把它们合并成更大的整数结果，最后再对目标模数取模。

这种方法的优点是：

每次 NTT 仍然用 `int / long long` 安全计算

缺点是：

要做多次 NTT，常数变大

4. 大模数还要满足 NTT 条件

NTT 不是任意模数都能直接做。长度为 n 的 NTT 要求存在 n 次单位根，因此通常需要：

$$n \mid (\text{mod} - 1)$$

如果 $n = 2^k$ ，就要求：

$$\text{mod} = c * 2^k + 1$$

例如：

$$998244353 = 119 * 2^{23} + 1$$

所以大模数下还有一个问题：**模数虽然大，但不一定适合 NTT。**

比如某个大质数 p ，如果 $p - 1$ 里面没有足够大的 2 的因子，那么它不能支持很长的二进制长度 NTT。

5. 原根计算和验证也更麻烦

NTT 需要原根 g ，长度为 n 时用：

$$\omega = g^{((\text{mod} - 1) / n) \bmod \text{mod}}$$

小模数下常见模数的原根基本固定，比如：

998244353 的原根是 3

但大模数下你可能需要自己找原根。找原根要求分解：

$$\text{mod} - 1$$

如果 mod 很大，分解 $\text{mod} - 1$ 本身也可能不容易。

6. 性能问题

大模数 NTT 每个蝶形里都有多次模乘。小模数可以非常快：

```
long long x = a * b % mod;
```

但大模数如果用：

```
(__int128)a * b % mod
```

会慢一些。

如果用 Montgomery 或 Barrett，可以优化，但实现复杂度会上升。

所以大模数 NTT 的性能瓶颈通常是：

模乘

而不是加减法或循环结构。

7. 逆变换也有同样的问题

逆 NTT 需要：

```
omega_inv  
n_inv
```

也就是：

```
omega_inv = pow_mod(omega, mod - 2, mod);  
n_inv = pow_mod(n, mod - 2, mod);
```

如果模数很大，快速幂中的每一步也需要安全模乘。

所以不能只修蝶形里的乘法，`qpow` 也要用 `mul_mod`。

总结

大模数 NTT 的难点主要有这些：

1. `a * b % mod` 会溢出
2. 需要 `__int128`、Barrett、Montgomery 或 CRT
3. 模数必须支持足够长的单位根
4. 找原根可能需要分解 `mod - 1`
5. 模乘变慢，性能常数变大
6. 正变换、逆变换、快速幂都要使用安全模乘

如果只是“模数超过 `int` 但还在 `long long` 内”，最简单做法是用：

```
using i128 = __int128_t;  
  
long long mul_mod(long long a, long long b, long long mod) {  
    return (long long)((i128)a * b % mod);  
}
```

如果目标模数不是 NTT-friendly，或者模数太大，那更常见的工程做法是：**多个小 NTT 模数 + CRT 合并**。

帮我详细介绍一下Montgomery规约以及他在NTT中的运用

Montgomery 规约的核心目标是：**把模乘里的除法/取模 $\% \text{ mod}$ 换成位运算和乘法**。在 NTT 里，它主要用来加速大量的：

$$a * b \% \text{ mod}$$

尤其当模数较大、或者 NTT 长度很大时，模乘会成为主要瓶颈。

1. 为什么需要 Montgomery

普通模乘：

$$c = a * b \% \text{ mod};$$

如果 mod 是 998244353 这种 32 位模数，`long long` 乘法后再 $\%$ 通常能跑得不错。

但 $\% \text{ mod}$ 本质上是除法，除法比加法、乘法、位运算慢很多。NTT 的每一层蝶形都要做大量模乘：

$$\begin{aligned} v &= a[i + j + \text{len} / 2] * w \% \text{ mod}; \\ w &= w * w_{\text{len}} \% \text{ mod}; \end{aligned}$$

所以如果能把 $\% \text{ mod}$ 优化掉，NTT 会明显变快。

Montgomery 规约做的事就是：

$$\text{快速计算 } a * b * R^{(-1)} \bmod \text{ mod}$$

其中 R 是一个 2 的幂，比如：

$$R = 2^{32} \quad \text{或} \quad R = 2^{64}$$

因为 R 是 2 的幂，所以对 R 取模和除以 R 都可以用位运算完成。

2. Montgomery 表示

Montgomery 不是直接表示整数 x ，而是把它表示成：

$$x_{\text{bar}} = x * R \bmod \text{mod}$$

这个 x_{bar} 叫 Montgomery form。

例如普通域里有：

$$x$$

Montgomery 域里存的是：

$$xR \bmod \text{mod}$$

如果两个数都在 Montgomery 域：

$$a_{\text{bar}} = aR \bmod \text{mod}$$

$$b_{\text{bar}} = bR \bmod \text{mod}$$

我们希望乘完以后仍然留在 Montgomery 域：

$$a_{\text{bar}} * b_{\text{bar}} = a b R^2$$

这时候 Montgomery 规约会额外乘一个 R^{-1} ：

$$\text{REDC}(a_{\text{bar}} * b_{\text{bar}})$$

$$= a b R^2 * R^{-1}$$

$$= a b R \bmod \text{mod}$$

结果仍然是 Montgomery form。

所以在 Montgomery 域中：

$$\text{mont_mul}(a_{\text{bar}}, b_{\text{bar}}) = (a * b)_{\text{bar}}$$

这非常适合 NTT，因为 NTT 里会连续做大量模乘，所有数一直放在 Montgomery 域里即可。

3. Montgomery 规约公式

设模数为：

mod

要求：

$$\gcd(\text{mod}, R) = 1$$

因为 R 是 2 的幂，所以通常要求：

mod 是奇数

这对 NTT 质数模数通常天然成立。

预处理一个常数：

$$\text{mod_inv} = -\text{mod}^{-1} \bmod R$$

也就是：

$$\text{mod} * \text{mod_inv} \equiv -1 \bmod R$$

Montgomery 规约 $\text{REDC}(T)$ 的目标是：

$$T * R^{-1} \bmod \text{mod}$$

步骤是：

```
m = (T mod R) * mod_inv mod R
t = (T + m * mod) / R
if t >= mod:
    t -= mod
return t
```

为什么这样可以？

因为：

$$\text{mod} * \text{mod_inv} \equiv -1 \pmod{R}$$

所以：

$$m * \text{mod} \equiv -(T \pmod{R}) \pmod{R}$$

因此：

$$T + m * \text{mod} \equiv 0 \pmod{R}$$

也就是说：

$$T + m * \text{mod}$$

一定能被 R 整除。

这样就避免了对 mod 的除法，只需要：

取低位、乘法、加法、右移

4. 一个小例子

为了方便手算，取：

$$\text{mod} = 17$$

$$R = 32$$

因为：

$$\text{gcd}(17, 32) = 1$$

先求：

$$\text{mod_inv} = -17^{(-1)} \pmod{32}$$

由于：

$$17 * 17 = 289 \equiv 1 \pmod{32}$$

所以：

$$17^{(-1)} \equiv 17 \pmod{32}$$

$$\text{mod_inv} \equiv -17 \equiv 15 \pmod{32}$$

现在计算普通意义下：

$$5 * 7 \pmod{17} = 35 \pmod{17} = 1$$

先进入 Montgomery 域：

$$5_bar = 5 * 32 \pmod{17} = 160 \pmod{17} = 7$$

$$7_bar = 7 * 32 \pmod{17} = 224 \pmod{17} = 3$$

做 Montgomery 乘法：

$$T = 5_bar * 7_bar = 7 * 3 = 21$$

规约：

$$m = (21 \pmod{32}) * 15 \pmod{32}$$

$$= 21 * 15 \pmod{32}$$

$$= 315 \pmod{32}$$

$$= 27$$

$$t = (21 + 27 * 17) / 32$$

$$= (21 + 459) / 32$$

$$= 480 / 32$$

$$= 15$$

得到：

它不是普通结果 1，而是 Montgomery 表示：

$$1_bar = 1 * 32 \bmod 17 = 15$$

所以结果正确。

如果要从 Montgomery 域转回普通整数，再做一次：

$$REDC(15) = 1$$

5. 在 NTT 中怎么用

普通 NTT 蝶形是：

```
u = a[i + j];  
v = a[i + j + len / 2] * w % mod;  
  
a[i + j] = u + v;  
a[i + j + len / 2] = u - v;
```

Montgomery 版本的思路是：

所有 a 都存成 Montgomery form
所有单位根 w 也存成 Montgomery form
乘法用 `mont_mul`
加减法仍然正常做模加模减

也就是说：

```
u = a[i + j]; // Montgomery form  
v = mont_mul(a[i + j + len / 2], w); // Montgomery form  
  
a[i + j] = add_mod(u, v);  
a[i + j + len / 2] = sub_mod(u, v);
```

注意，加法减法不需要特殊处理。因为：

$$a_bar + b_bar = aR + bR = (a + b)R$$

所以 Montgomery 域对加减法是天然兼容的。

单位根更新也用：

```
w = mont_mul(w, wlen);
```

其中 `w` 和 `wlen` 都是 Montgomery form。

6. NTT 的整体流程

假设输入数组是普通整数：

```
a[i]
```

第一步：转入 Montgomery 域：

```
a[i] = to_mont(a[i]);
```

单位根也转入：

```
root = to_mont(root);  
root_inv = to_mont(root_inv);
```

然后正常跑 NTT，只是把乘法换成：

```
mont_mul(x, y)
```

正变换结束后，如果只是要继续做点值乘法，可以不转回普通域。

比如多项式乘法：


```
A -> NTT
B -> NTT
C[i] = A[i] * B[i]
inverse NTT
```

在 Montgomery 域中可以这样：

```
ntt(A);
ntt(B);

for i:
    C[i] = mont_mul(A[i], B[i]);

intt(C);
```

逆 NTT 结束后，还需要乘 n_{inv} 。如果 n_{inv} 也在 Montgomery 域：

```
C[i] = mont_mul(C[i], n_inv_mont);
```

最后转回普通整数：

```
C[i] = from_mont(C[i]);
```

7. C++ 形态示意：32 位模数

对于 $\text{mod} < 2^{31}$ 或常见 NTT 模数，可以用 `uint64_t` 承接乘积， $R = 2^{32}$ 。

```

struct Montgomery {
    using u32 = uint32_t;
    using u64 = uint64_t;

    u32 mod;
    u32 inv; // inv = -mod^{-1} mod 2^32
    u32 r2; // R^2 mod mod

    Montgomery(u32 mod) : mod(mod) {
        inv = get_inv32(mod);
        r2 = (u64(-1) % mod + 1) % mod; // 2^32 mod mod
        r2 = (u64)r2 * r2 % mod; // R^2 mod mod
    }

    u32 reduce(u64 x) const {
        u32 m = u32(x) * inv;
        u64 t = (x + (u64)m * mod) >> 32;
        if (t >= mod) t -= mod;
        return u32(t);
    }

    u32 mul(u32 a, u32 b) const {
        return reduce((u64)a * b);
    }

    u32 init(u32 x) const {
        return mul(x, r2); // x * R^2 * R^{-1} = xR
    }

    u32 get(u32 x) const {
        return reduce(x); // xR * R^{-1} = x
    }
};

```

这里最容易困惑的是 `init(x)` 为什么乘 R^2 ：

$$\text{REDC}(x * R^2) = xR$$

所以它能把普通数转成 Montgomery form。

8.64 位模数怎么办

如果 mod 接近 2^{63} ，可以取：

$$R = 2^{64}$$

这时乘积 $a * b$ 需要 128 位保存：

`__uint128_t`

规约时会出现：

$$x + m * \text{mod}$$

这个值理论上可能需要 129 位，但常见实现会利用无符号溢出性质或拆高低位来做。实现细节比 32 位版本更绕。

如果只是 $\text{mod} < 2^{63}$ ，很多场景下：

$$(_\text{int128})a * b \% \text{mod}$$

已经足够简单可靠；Montgomery 更适合追求性能的实现。

9. Montgomery 在 NTT 里的优势

主要优势：

1. 加速模乘，减少慢速除法 `%`
2. 所有蝶形中的乘法都可以统一使用 `mont_mul`
3. 加减法和普通模运算一样简单
4. 非常适合固定模数的高性能 NTT
5. 可配合 `lazy reduction` 进一步减少取模次数

尤其对于这种高频操作：

$$\begin{aligned} v &= a[k] * w \% \text{mod}; \\ w &= w * \text{wlen} \% \text{mod}; \end{aligned}$$

Montgomery 可以把 $\% \text{ mod}$ 换成更快的规约。

10. 在蝶形中配合 lazy reduction

NTT 中常见优化是让数暂时落在 $[0, 2\text{mod})$ ，不每次都完全规约到 $[0, \text{mod})$ 。

普通写法：

```
x = u + v;  
if (x >= mod) x -= mod;  
  
y = u - v;  
if (y < 0) y += mod;
```

如果用无符号整数，可以写成：

```
u32 add(u32 a, u32 b) {  
    u32 c = a + b;  
    if (c >= mod) c -= mod;  
    return c;  
}  
  
u32 sub(u32 a, u32 b) {  
    return a >= b ? a - b : a + mod - b;  
}
```

Montgomery 只负责乘法规约，加减法仍然用这些普通模加减。

11. 常见坑

Montgomery 用在 NTT 里，最常见的坑有：

1. 忘记把输入转成 Montgomery form
2. 忘记把单位根转成 Montgomery form
3. 逆变换后的 `n_inv` 没有用 Montgomery form
4. 最后忘记 `from_mont`
5. `mod` 必须和 `R` 互质，也就是 `mod` 必须是奇数
6. `reduce` 的 `inv` 是 $-\text{mod}^{-1} \bmod R$ ，不是 mod^{-1}
7. 32 位和 64 位版本不能随便混用

一句话总结

Montgomery 规约通过把数存成 $xR \bmod \text{mod}$ ，让模乘：

$$a * b \bmod \text{mod}$$

变成：

$$\text{REDC}(a_{\text{bar}} * b_{\text{bar}})$$

从而避免昂贵的除法取模。NTT 里大量出现模乘，所以把数组、单位根、逆元都放进 Montgomery 域后，蝶形结构不变，只需要把乘法替换成 `mont_mul`，最后再转回普通整数即可。

我现在实现了montgomery规约取模，现在我想把NTT蝶形变换中的核心部分使用SIMD NEON进行优化，我目前的思路是进行4路并行，在加减中直接向量加减即可，但是在向量相乘的部分32位与32位相乘会出现64位这样就无法4路并行了，如果我想实现这种乘法的并行我应该怎么办

你的方向是对的，但这里有一个 SIMD 的“宽度现实”：**128-bit NEON 装得下 4 个 `uint32_t`，但装不下 4 个 `uint64_t`**。所以 $32 \times 32 \rightarrow 64$ 的乘法不能在一个寄存器里保留 4 路 64-bit 结果。

常用做法是：**4 路逻辑并行**，内部拆成低 2 路和高 2 路分别做 64-bit Montgomery 规约，然后再窄化合并回 `uint32x4_t`。

也就是：

```
uint32x4_t a = [a0, a1, a2, a3]
uint32x4_t b = [b0, b1, b2, b3]
```

低半部分：

```
[a0*b0, a1*b1] -> uint64x2_t
```

高半部分：

```
[a2*b2, a3*b3] -> uint64x2_t
```

分别 Montgomery reduce

再合成：

```
[r0, r1, r2, r3] -> uint32x4_t
```

1. NEON 里应该这样做乘法

NEON 提供 widening multiply：

```
vmull_u32
```

它一次做：

```
uint32x2_t * uint32x2_t -> uint64x2_t
```

所以 4 路乘法写法是：

```
uint64x2_t t0 = vmull_u32(vget_low_u32(a), vget_low_u32(b));
uint64x2_t t1 = vmull_u32(vget_high_u32(a), vget_high_u32(b));
```

如果是 AArch64，也可以用：

```
uint64x2_t t0 = vmull_u32(vget_low_u32(a), vget_low_u32(b));
uint64x2_t t1 = vmull_high_u32(a, b);
```

本质一样。

2. 32-bit Montgomery 规约的 SIMD 版本

假设你的 Montgomery 是：

$$R = 2^{32}$$
$$\text{inv} = -\text{mod}^{-1} \bmod 2^{32}$$

标量版通常是：

```
uint32_t reduce(uint64_t x) {  
    uint32_t m = uint32_t(x) * inv;  
    uint64_t t = (x + uint64_t(m) * mod) >> 32;  
    if (t >= mod) t -= mod;  
    return uint32_t(t);  
}
```

对应 NEON 版本可以写成这样：

```
#include <arm_neon.h>
```

```
struct Mont32Neon {  
    uint32_t mod;  
    uint32_t inv; // inv = -mod^{-1} mod 2^32
```

```
    uint32x4_t mod4;  
    uint32x2_t mod2;  
    uint32x2_t inv2;
```

```
Mont32Neon(uint32_t mod_, uint32_t inv_)  
    : mod(mod_), inv(inv_) {  
    mod4 = vdupq_n_u32(mod);  
    mod2 = vdup_n_u32(mod);  
    inv2 = vdup_n_u32(inv);  
}
```

```
uint32x4_t reduce_u64x2_pair(uint64x2_t x0, uint64x2_t x1) const {  
    // m = low32(x) * inv mod 2^32  
    uint32x2_t m0 = vmul_u32(vmovn_u64(x0), inv2);  
    uint32x2_t m1 = vmul_u32(vmovn_u64(x1), inv2);  
  
    // u = x + m * mod  
    uint64x2_t u0 = vmlal_u32(x0, m0, mod2);  
    uint64x2_t u1 = vmlal_u32(x1, m1, mod2);  
  
    // t = u >> 32  
    uint32x2_t r0 = vshrn_n_u64(u0, 32);  
    uint32x2_t r1 = vshrn_n_u64(u1, 32);  
  
    uint32x4_t r = vcombine_u32(r0, r1);  
  
    // if (r >= mod) r -= mod  
    uint32x4_t r_sub = vsubq_u32(r, mod4);  
    uint32x4_t mask = vcgeq_u32(r, mod4);  
    return vbslq_u32(mask, r_sub, r);  
}
```

```
uint32x4_t mul(uint32x4_t a, uint32x4_t b) const {  
    uint64x2_t x0 = vmull_u32(vget_low_u32(a), vget_low_u32(b));  
    uint64x2_t x1 = vmull_u32(vget_high_u32(a), vget_high_u32(b));  
    return reduce_u64x2_pair(x0, x1);  
}
```



```
    }  
};
```

这样你虽然中间用了两个 `uint64x2_t`，但整体接口仍然是：

```
uint32x4_t mont_mul(uint32x4_t a, uint32x4_t b);
```

所以在 NTT 蝶形里还是 4 路处理。

3. 用在 NTT 蝶形里

标量蝶形：

```
u = a[j];  
v = mont_mul(a[j + half], w);
```

```
a[j]          = u + v;  
a[j + half] = u - v;
```

NEON 4 路版本：

```
uint32x4_t add_mod(uint32x4_t a, uint32x4_t b, uint32x4_t mod) {
    uint32x4_t s = vaddq_u32(a, b);
    uint32x4_t s_sub = vsubq_u32(s, mod);
    uint32x4_t mask = vcgeq_u32(s, mod);
    return vbslq_u32(mask, s_sub, s);
}
```

```
uint32x4_t sub_mod(uint32x4_t a, uint32x4_t b, uint32x4_t mod) {
    uint32x4_t d1 = vsubq_u32(a, b);
    uint32x4_t d2 = vsubq_u32(vaddq_u32(a, mod), b);
    uint32x4_t mask = vcgeq_u32(a, b);
    return vbslq_u32(mask, d1, d2);
}
```

```
void butterfly4(
    uint32_t* a,
    int j,
    int half,
    uint32x4_t w,
    const Mont32Neon& mt
) {
    uint32x4_t u = vld1q_u32(a + j);
    uint32x4_t x = vld1q_u32(a + j + half);

    uint32x4_t v = mt.mul(x, w);

    uint32x4_t y0 = add_mod(u, v, mt.mod4);
    uint32x4_t y1 = sub_mod(u, v, mt.mod4);

    vst1q_u32(a + j, y0);
    vst1q_u32(a + j + half, y1);
}
```

这里的 w 应该是 4 个连续蝶形对应的旋转因子：

$$w = [w_j, w_{j+1}, w_{j+2}, w_{j+3}]$$

并且它们也必须已经是 Montgomery form。

4. 旋转因子怎么处理

最推荐的做法是：**预计算每一层的 twiddle，并且直接存 Montgomery form。**

比如某一层 len ，每组内需要：

$1, wlen, wlen^2, wlen^3, \dots$

你可以预先存成连续数组：

```
roots[j + 0]
roots[j + 1]
roots[j + 2]
roots[j + 3]
```

然后 SIMD 里直接：

```
uint32x4_t w = vld1q_u32(roots + j);
```

这样比每次在循环里更新 w 更适合 SIMD，因为标量更新：

```
w *= wlen;
```

有链式依赖，会限制吞吐。

如果你不想预计算，也可以每次构造：

```
[w, w*wlen, w*wlen^2, w*wlen^3]
```

然后每轮把整个向量乘上：

```
wlen^4
```

但实现复杂一些，通常不如预计算干净。

5. 小长度层怎么处理

当：

$$\text{half} < 4$$

也就是 $\text{len} = 2$ 或 $\text{len} = 4$ 时，正常的“连续 4 个 j”不够用。

常见处理方式：

$\text{len} = 2, 4$ ：用标量或专门写小 kernel

$\text{len} \geq 8$ ：用 4 路 NEON

这很正常。NTT 的大头在后面的长层，小层用标量影响通常不大。

6. 一个重要限制：mod 最好小于 2^{31}

上面这个 Montgomery NEON 写法默认：

$$\text{mod} < 2^{31}$$

这对大多数 32-bit NTT 质数都成立，比如：

998244353

1004535809

469762049

原因是这里用了：

$$u = x + m * \text{mod}$$

然后直接右移 32 位。如果 mod 接近 2^{32} ， $x + m * \text{mod}$ 可能超过 64 位，需要额外处理 carry。那样 SIMD 规约会复杂不少。

所以如果你的模数是 32-bit NTT prime，建议尽量选：

$$\text{mod} < 2^{31}$$

这会让 NEON Montgomery 非常舒服。

7. 如果你的模数真的超过 32 位

如果 mod 是 64-bit，比如接近 $1e18$ ，那就不是这个路线了。

128-bit NEON 里：

```
uint64x2_t
```

只能放 2 路 64-bit，而且常规 NEON 没有好用的：

```
uint64 * uint64 -> uint128
```

向量乘法。

这时常见选择是：

1. 放弃 4 路 64-bit SIMD，使用标量 `__int128`
2. 用多个 30/31-bit NTT 模数 + CRT
3. 换 SVE / 特定平台指令
4. 使用更复杂的拆 limb 乘法，但通常不划算

对于 NTT，高性能实现一般更偏向：

```
多个 32-bit friendly primes + SIMD NTT + CRT
```

而不是直接做 64-bit 大模数 SIMD NTT。

结论

你不需要避免 $32 \times 32 \rightarrow 64$ 。正确做法是：

```
uint32x4_t
```

```
拆成两个 uint32x2_t
```

```
分别 vmull_u32 得到两个 uint64x2_t
```

```
分别 Montgomery reduce
```

```
再 vcombine_u32 合回 uint32x4_t
```

也就是说，NEON 中的 4 路 Montgomery 乘法通常是“两个 2 路 widening multiply + 两个 2 路 reduce”，外部看起来仍然是 4 路并行。NTT 蝶形里的加减继续用 `uint32x4_t` 做，乘法用这个 `mont_mul4` 替换即可。

介绍一下DIF的原理

DIF，全称 **Decimation in Frequency**，中文一般叫 **按频率抽取**。它是 FFT/NTT 的一种迭代实现方式。

和常见的 DIT（Decimation in Time，按时间抽取）相比，DIF 的核心区别是：

DIT：先乘旋转因子，再做加减

DIF：先做加减，再乘旋转因子

1. 从多项式取值看 DIF

设长度为 N 的 NTT 要计算：

$$A(\omega^0), A(\omega^1), \dots, A(\omega^{N-1})$$

其中 ω 是 N 次单位根。

把频域结果按照偶数频率和奇数频率分开：

偶数频率： $A(\omega^0), A(\omega^2), A(\omega^4), \dots$

奇数频率： $A(\omega^1), A(\omega^3), A(\omega^5), \dots$

DIF 的名字“按频率抽取”就是因为它从频域下标的奇偶性出发，把输出频率拆成两半。

2. DIF 的拆分方式

设多项式：

$$A(x) = \sum_{j=0}^{N-1} a_j x^j$$

把系数按前半和后半拆开：

$$A(x) = \sum_{j=0}^{N/2-1} a_j x^j + \sum_{j=0}^{N/2-1} a_{j+N/2} x^{j+N/2}$$

对 $x = \omega^k$ ，有：

$$A(\omega^k) = \sum_{j=0}^{N/2-1} (a_j + a_{j+N/2} \omega^{kN/2}) \omega^{kj}$$

因为：

$$\omega^{N/2} = -1$$

所以：

$$\omega^{kN/2} = (-1)^k$$

于是当 k 为偶数时：

$$A(\omega^{2r}) = \sum_{j=0}^{N/2-1} (a_j + a_{j+N/2}) \omega^{2rj}$$

当 k 为奇数时：

$$A(\omega^{2r+1}) = \sum_{j=0}^{N/2-1} (a_j - a_{j+N/2}) \omega^{(2r+1)j}$$

又因为：

$$\omega^{(2r+1)j} = \omega^j (\omega^2)^{rj}$$

所以：

$$A(\omega^{2r+1}) = \sum_{j=0}^{N/2-1} ((a_j - a_{j+N/2}) \omega^j) (\omega^2)^{rj}$$

这说明：

偶数频率结果来自 $a[j] + a[j+N/2]$

奇数频率结果来自 $(a[j] - a[j+N/2]) * \omega^j$

这就是 DIF 的蝶形公式。

3. DIF 蝶形公式

令：

$$u = a_j$$

$$v = a_{j+N/2}$$

那么 DIF 蝶形为：

$$a'_j = u + v$$

$$a'_{j+N/2} = (u - v)\omega^j$$

在模 p 的 NTT 中写成：

$$a'_j = (u + v) \bmod p$$

$$a'_{j+N/2} = (u - v) \cdot \omega^j \bmod p$$

对应代码就是：

```
uint32_t u = f[i + j];
uint32_t v = f[i + j + half];

f[i + j] = mont.add(u, v);
f[i + j + half] = mont.mul(mont.sub(u, v), w);

w = mont.mul(w, wn);
```

4. DIF 的迭代方向

DIF 一般从大区间开始拆：


```
len = N
len = N/2
len = N/4
...
len = 2
```

也就是代码结构：

```
for (int len = n; len >= 2; len >>= 1) {
    int half = len >> 1;
    ...
}
```

这和 DIT 正好相反。

DIT 是：

```
len = 2
len = 4
len = 8
...
len = N
```

5. DIF 的输入输出顺序

DIF 的一个重要特点是：

输入是自然顺序
输出通常是位翻转顺序

而 DIT 通常是：

输入先位翻转
输出自然顺序

因此，可以用：

DIF 正变换 + DIT 逆变换

来减少显式 bit reversal。

流程是：

A 自然序 -> DIF NTT -> A 位翻转频域

B 自然序 -> DIF NTT -> B 位翻转频域

逐点相乘

位翻转频域 -> DIT inverse NTT -> 自然序结果

因为两个多项式经过 DIF 后都是相同的位翻转顺序，所以逐点相乘仍然是对应频点相乘。

6. 和 DIT 的对比

DIT 蝶形：

$$v = w \cdot right$$

$$left' = u + v$$

$$right' = u - v$$

对应代码：

```
v = mont.mul(w, f[i + j + half]);
```

```
f[i + j] = mont.add(u, v);
```

```
f[i + j + half] = mont.sub(u, v);
```

DIF 蝶形：

$$left' = u + right$$

$$right' = (u - right) \cdot w$$

对应代码：

```
f[i + j] = mont.add(u, v);  
f[i + j + half] = mont.mul(mont.sub(u, v), w);
```

区别就是：

DIT：旋转因子乘在加减之前

DIF：旋转因子乘在加减之后

一句话总结：

DIF 是从频率下标的奇偶性出发，把一个长度为 N 的 NTT 拆成两个长度为 $N/2$ 的 NTT。它的蝶形操作是先计算 $u + v$ 和 $u - v$ ，再让差值乘旋转因子。它通常自然序输入、位翻转序输出，适合和 DIT 逆变换配合来减少显式位翻转。

我要实现大模数263882790666241的NTT，我初步的思路是使用64位的规约取模，然后两路并行，这有什么问题吗

你的思路总体是对的，但需要把“哪些部分两路并行”说清楚。

对于大模数：

263882790666241

它小于 2^{64} ，所以可以用 `uint64_t` 存储；但模乘中间结果是：

$$uint64_t \times uint64_t \rightarrow uint128_t$$

因此你需要：

`__uint128_t`

或者 Montgomery64 来保证模乘正确。

可行部分

你可以实现：

```
Montgomery64 规约取模  
NEON uint64x2_t 两路蝶形加减
```

也就是：

```
v0 = mont.mul(w0, right0); // 标量 Montgomery64  
v1 = mont.mul(w1, right1); // 标量 Montgomery64  
  
[u0, u1] + [v0, v1]          // NEON 两路  
[u0, u1] - [v0, v1]          // NEON 两路
```

这个思路是可行的，也比较稳。

主要问题

真正困难的是：**两路 Montgomery 模乘不容易 SIMD 化。**

因为 32 位版本里：

```
uint32_t * uint32_t -> uint64_t
```

NEON 有比较好用的：

```
vmull_u32
```

可以一次做两路并扩展成 `uint64x2_t`。

但大模数下需要：

```
uint64_t * uint64_t -> uint128_t
```

NEON 没有直接好用的：

```
uint64x2_t -> uint128x2_t
```

所以你不能像 32 位版本那样自然写一个：

montgomery_mul2_neon_u64

如果要真正 SIMD 化 64 位 Montgomery 模乘，需要把每个 64 位数拆成高低 32 位，手工组合 128 位乘积，还要处理进位和规约，代码复杂度很高。

所以推荐表述为

你的方案应写成：

使用 Montgomery64 处理大模数模乘；

由于大模数乘法需要 128 位中间结果，模乘部分采用标量 __uint128_t；

在蝶形变换中，对模加和模减使用 uint64x2_t 做两路 NEON 并行。

这就没有问题。

需要注意的点

1. 输入、数组、模数都要用 64 位：

```
uint64_t a[], b[], ab[];
uint64_t p;
```

2. 读入时如果担心负数，可以用 long long 临时读入，再归一化到 $[0, p)$ 。
3. 不能用 int p，否则大模数一读入就溢出。
4. 不能继续用 Montgomery32，要用：

Montgomery64

5. 逆 NTT 最后乘 n^{-1} 也要用 Montgomery64。
6. 两路 NEON 加减是可以做的，但性能提升会有限，因为最重的模乘仍然是标量。

总结

你的思路没有方向性问题，但更准确地说应该是：

大模数 NTT = Montgomery64 标量模乘 + NEON 两路蝶形加减

而不是：

大模数 NTT = 两路并行 Montgomery 模乘

后者能做，但实现难度明显高很多。